Universidad Privada del Estado de México.

Quinto Cuatrimestre.

Ingeniería en Sistemas Computacionales.

Programación Avanzada

Proyecto

Sistema de Agenda de Contactos

Profesor. Ing. Neri Alejandro Alvarez Esperon

Alumno: Edgar Antonio Mendoza

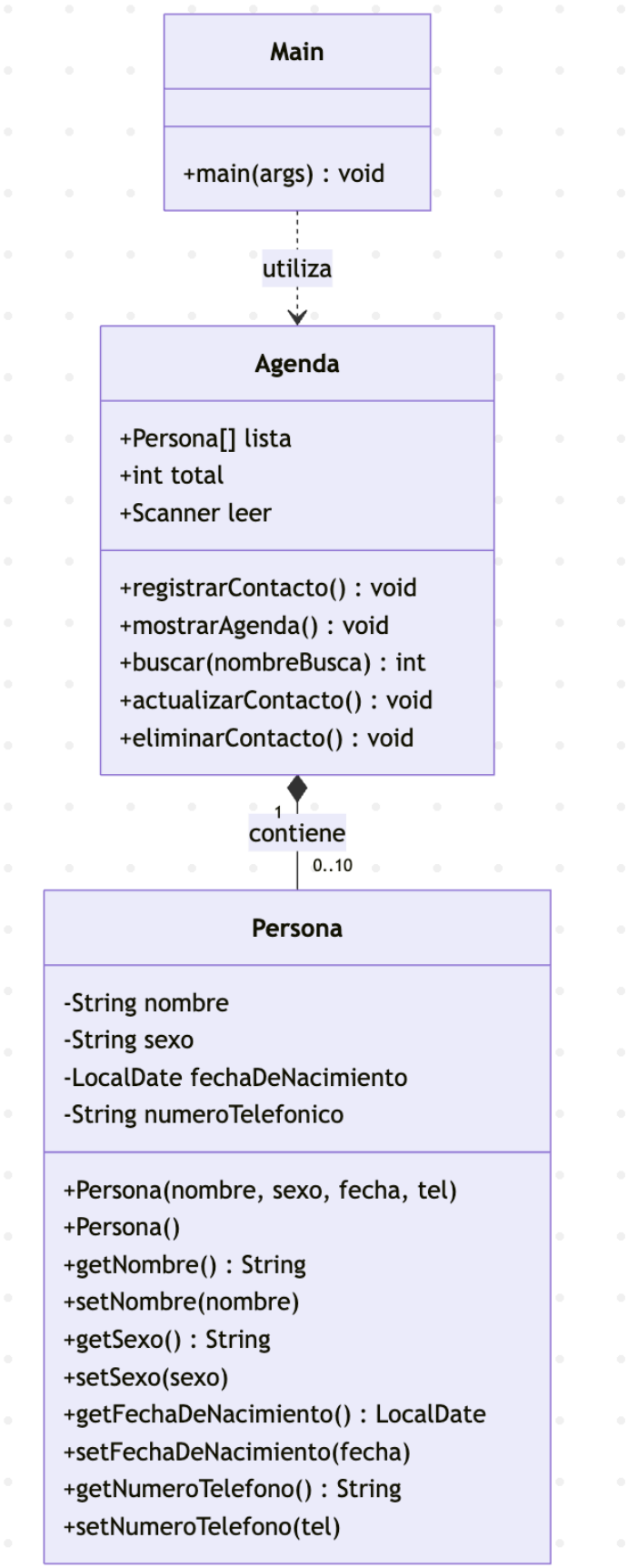
Marzo 2026

Introducción

En este proyecto, desarrollamos una Agenda de Contactos aplicando los conceptos que aprendimos sobre la Programación Orientada a Objetos (POO). Para programar el sistema, elegimos el lenguaje Java, ya que nos permite organizar la información de manera clara a través de clases y objetos.

El programa está diseñado para administrar datos personales como el nombre, sexo, fecha de nacimiento y teléfono de cada contacto. Para lograr esto, creamos una clase con sus respectivos atributos, un método constructor y los métodos getters y setters para manejar la información. Además, utilizamos un arreglo (lista) como estructura principal para guardar todos los registros en la memoria del programa.

Para que el sistema sea fácil de usar, implementamos un menú interactivo en la consola. Mediante este menú, cualquier usuario puede navegar por las diferentes opciones de la agenda, tales como registrar nuevas personas, buscar a alguien en específico, actualizar datos que hayan cambiado, borrar registros o simplemente ver la lista completa de contactos almacenados.


Main
+main(args) : void
utiliza
Agenda
+Persona[] lista
+int total
+Scanner leer
+registrarContacto() : void
+mostrarAgenda() : void
+buscar(nombreBusca) : int
+actualizarContacto() : void
+eliminarContacto() : void
1
contiene
0..10
Persona
-String nombre
-String sexo
-LocalDate fechaDeNacimiento
-String numeroTelefonico
+Persona(nombre, sexo, fecha, tel)
+Persona()
+getNombre() : String
+setNombre(nombre)
+getSexo() : String
+setSexo(sexo)
+getFechaDeNacimiento() : LocalDate
+setFechaDeNacimiento(fecha)
+getNumeroTelefono() : String
+setNumeroTelefono(tel)

Descripción del Sistema

El programa funciona mediante una interfaz de consola donde el usuario elige opciones de un menú para gestionar hasta 10 registros. El diseño se divide en tres partes fundamentales:

1. **Clase Persona:** Es el modelo de los datos. Aquí se define qué información guardamos de cada persona (nombre, sexo, fecha y teléfono) usando variables privadas.

2. **Clase Agenda:** Es el motor del sistema. Aquí se crea el arreglo para guardar los objetos y se programan todas las funciones lógicas como añadir, buscar, editar o borrar contactos.

3. **Clase Main:** Es el punto de inicio. Contiene el ciclo principal y el menú que recibe las órdenes del usuario para conectar las acciones con la lógica de la Agenda.

Explicación del Código
Descripción de cada clase:

- Persona: Define las características del contacto.
- Agenda: Administra el almacenamiento y las acciones (CRUD).
- Main: Controla el flujo de entrada y salida de datos por consola.

Uso del constructor: Se utiliza para inicializar los objetos Persona con valores específicos (nombre, tel, etc.) al momento de crearlos con la palabra new. En el código tenemos dos: uno que recibe datos y otro por "default" con valores ya fijos.

Encapsulamiento: Se declararon los atributos como private por seguridad. Esto evita que los datos se modifiquen por error desde fuera de la clase, obligando a usar los métodos getters y setters para leer o cambiar la información de forma controlada.

Menú y flujo del programa: El flujo es lineal-repetitivo. El Main muestra las opciones, el usuario elige un número y, mediante condicionales if-else, el programa "salta" al método correspondiente en la clase Agenda. Al terminar la acción, el flujo regresa al menú principal.

Conclusión

La verdad, me pareció súper interesante ver cómo se puede armar un CRUD completo en Java. Siendo sincero, al principio las prácticas se sentían muy básicas, pero este proyecto, aunque parezca sencillo, sí tuvo su reto. Lo más complicado para mí fue volver a entender bien el tema de los arreglos y listas; me costó trabajo recordar la lógica para eliminar un elemento sin que el programa fallara o cómo usar los bucles para recorrer todo sin perderme.

Creo que nos vendrían bien más prácticas de este tipo, porque sí estuvo difícil lograr que todo conectara a la primera. Por ejemplo, me quebré la cabeza buscando la mejor forma de hacer las búsquedas para que el sistema encontrará a las personas de la agenda sin importar si escribía en mayúsculas o minúsculas. Al final se logró, pero sí fue un proceso de mucho intentar y corregir.